# CRUD de Produtos e Categorias

Java Swing + JDBC + MySQL no NetBeans, com separação em pacotes (modelo, dao, util, visao)

## 1. Visão geral

Sistema de controle de estoque com duas tabelas relacionadas: **Categorias** e **Produtos** (com chave estrangeira `categoria_id`). Requisitos atendidos:

- Cadastro de produtos com **seleção da categoria** (JComboBox preenchido com as categorias cadastradas).
- CRUD completo de produtos (criar, listar, atualizar e excluir). A categoria pode ser alterada na edição.
- Tela que exibe todos os produtos: nome, preço, quantidade e categoria.
- Tela separada para cadastrar, editar e listar categorias.
- Busca de produtos por **nome ou categoria**.

## 2. Como rodar no NetBeans

- Execute o `schema.sql` (seção 4) no MySQL (Workbench, phpMyAdmin ou terminal).
- NetBeans: *File* > *New Project* > *Java with Ant* > *Java Application* (desmarque *Create Main Class*).
- Crie os pacotes em *Source Packages* > *New* > *Java Package*: `loja`, `loja.modelo`, `loja.dao`, `loja.util` e `loja.visao`.
- Adicione o driver: botão direito em *Libraries* > *Add Library* > *MySQL JDBC Driver* (ou *Add JAR/Folder* com o mysql-connector-j). Sem ele ocorre o erro "No suitable driver".
- Ajuste `USUARIO` e `SENHA` em `Conexao.java`.
- Botão direito em `Main.java` > *Run File*.

## 3. Estrutura de pacotes

```
Source Packages
`-- loja
    |-- Main.java
    |-- modelo
    |   |-- Categoria.java
    |   `-- Produto.java
    |-- dao
    |   |-- CategoriaDAO.java
    |   `-- ProdutoDAO.java
    |-- util
    |   `-- Conexao.java
    `-- visao
        |-- TelaPrincipal.java
        |-- TelaCategorias.java
        `-- TelaProdutos.java
```

Cada camada só depende da anterior: **visao** usa **dao** e **modelo**; **dao** usa **modelo** e **util**; **modelo** não depende de nada. **util** concentra a conexão com o banco; **Main** apenas abre a primeira tela.

| Pacote | Responsabilidade |
|---|---|
| `loja.modelo` | Classes de dados (Categoria, Produto): atributos, construtor e getters. |
| `loja.dao` | Acesso ao banco: comandos SQL com PreparedStatement (inserir, atualizar, excluir, listar). |
| `loja.util` | Conexao: abre a conexão JDBC com o MySQL. |
| `loja.visao` | Telas Swing (JFrame): formulários, tabelas, botões e eventos. |
| `loja` | Main: ponto de entrada do programa. |

# 4. Banco de dados (SQL)

**schema.sql**

```
CREATE DATABASE IF NOT EXISTS loja CHARACTER SET utf8mb4;
USE loja;

CREATE TABLE IF NOT EXISTS Categorias (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS Produtos (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    nome         VARCHAR(150)  NOT NULL,
    preco        DECIMAL(10,2) NOT NULL,
    quantidade   INT           NOT NULL,
    categoria_id INT           NOT NULL,
    CONSTRAINT fk_produto_categoria
        FOREIGN KEY (categoria_id) REFERENCES Categorias(id)
);
```

# 5. Código-fonte

## 5.1 Pacote loja.util

**loja/util/Conexao.java**

```
package loja.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexao {
    private static final String URL = "jdbc:mysql://localhost:3306/loja?useSSL=false&serverTimezone=UTC";
    private static final String USUARIO = "root";
    private static final String SENHA = ""; // ajuste para a sua senha

    public static Connection getConexao() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, SENHA);
    }
}
```

## 5.2 Pacote loja.modelo

**loja/modelo/Categoria.java**

```
package loja.modelo;

public class Categoria {
    private int id;
    private String nome;

    public Categoria(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public int getId() { return id; }
    public String getNome() { return nome; }

    @Override
    public String toString() { return nome; } // usado pelo JComboBox
}
```

**loja/modelo/Produto.java**

```
package loja.modelo;

public class Produto {
    private int id;
    private String nome;
    private double preco;
    private int quantidade;
    private int categoriaId;
    private String categoriaNome;

    public Produto(int id, String nome, double preco, int quantidade, int categoriaId, String categoriaNome) {
        this.id = id;
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
        this.categoriaId = categoriaId;
        this.categoriaNome = categoriaNome;
    }

    public int getId() { return id; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }
    public int getQuantidade() { return quantidade; }
    public int getCategoriaId() { return categoriaId; }
    public String getCategoriaNome() { return categoriaNome; }
}
```

## 5.3 Pacote loja.dao

**loja/dao/CategoriaDAO.java**

```
package loja.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import loja.modelo.Categoria;
import loja.util.Conexao;

public class CategoriaDAO {

    public void inserir(Categoria c) throws SQLException {
        String sql = "INSERT INTO Categorias (nome) VALUES (?)";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, c.getNome());
            ps.executeUpdate();
        }
    }

    public void atualizar(Categoria c) throws SQLException {
        String sql = "UPDATE Categorias SET nome = ? WHERE id = ?";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, c.getNome());
            ps.setInt(2, c.getId());
            ps.executeUpdate();
        }
    }

    public void excluir(int id) throws SQLException {
        String sql = "DELETE FROM Categorias WHERE id = ?";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }

    public List<Categoria> listar() throws SQLException {
        List<Categoria> lista = new ArrayList<>();
        String sql = "SELECT id, nome FROM Categorias ORDER BY nome";
        try (Connection con = Conexao.getConexao();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                lista.add(new Categoria(rs.getInt("id"), rs.getString("nome")));
            }
        }
        return lista;
    }
}
```

**loja/dao/ProdutoDAO.java**

```
package loja.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import loja.modelo.Produto;
import loja.util.Conexao;

public class ProdutoDAO {

    public void inserir(Produto p) throws SQLException {
        String sql = "INSERT INTO Produtos (nome, preco, quantidade, categoria_id) VALUES (?, ?, ?, ?)";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, p.getNome());
            ps.setDouble(2, p.getPreco());
            ps.setInt(3, p.getQuantidade());
            ps.setInt(4, p.getCategoriaId());
            ps.executeUpdate();
        }
    }

    public void atualizar(Produto p) throws SQLException {
        String sql = "UPDATE Produtos SET nome = ?, preco = ?, quantidade = ?, categoria_id = ? WHERE id = ?";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, p.getNome());
            ps.setDouble(2, p.getPreco());
            ps.setInt(3, p.getQuantidade());
            ps.setInt(4, p.getCategoriaId());
            ps.setInt(5, p.getId());
            ps.executeUpdate();
        }
    }

    public void excluir(int id) throws SQLException {
        String sql = "DELETE FROM Produtos WHERE id = ?";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }

    /** Lista os produtos; se "busca" tiver texto, filtra por nome do produto OU da categoria. */
    public List<Produto> listar(String busca) throws SQLException {
        List<Produto> lista = new ArrayList<>();
        String sql = "SELECT p.id, p.nome, p.preco, p.quantidade, p.categoria_id, c.nome AS categoria_nome "
                   + "FROM Produtos p JOIN Categorias c ON c.id = p.categoria_id "
                   + "WHERE p.nome LIKE ? OR c.nome LIKE ? ORDER BY p.nome";
        String filtro = "%" + (busca == null ? "" : busca.trim()) + "%";
        try (Connection con = Conexao.getConexao(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, filtro);
            ps.setString(2, filtro);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    lista.add(new Produto(
                        rs.getInt("id"),
                        rs.getString("nome"),
                        rs.getDouble("preco"),
                        rs.getInt("quantidade"),
                        rs.getInt("categoria_id"),
                        rs.getString("categoria_nome")));
                }
            }
        }
        return lista;
    }
}
```

## 5.4 Pacote loja.visao

**loja/visao/TelaPrincipal.java**

```
package loja.visao;

import java.awt.GridLayout;
import javax.swing.JButton;
import javax.swing.JFrame;

public class TelaPrincipal extends JFrame {

    public TelaPrincipal() {
        setTitle("Controle de Estoque");
        setSize(300, 160);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new GridLayout(2, 1, 10, 10));

        JButton btnProdutos = new JButton("Produtos");
        JButton btnCategorias = new JButton("Categorias");
        btnProdutos.addActionListener(e -> new TelaProdutos().setVisible(true));
        btnCategorias.addActionListener(e -> new TelaCategorias().setVisible(true));

        add(btnProdutos);
        add(btnCategorias);
    }
}
```

**loja/visao/TelaCategorias.java**

```
package loja.visao;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.sql.SQLException;
import java.util.List;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.table.DefaultTableModel;
import loja.dao.CategoriaDAO;
import loja.modelo.Categoria;

public class TelaCategorias extends JFrame {

    private final CategoriaDAO dao = new CategoriaDAO();
    private final JTextField txtNome = new JTextField(20);
    private final DefaultTableModel modelo = new DefaultTableModel(new Object[]{"ID", "Nome"}, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };
    private final JTable tabela = new JTable(modelo);
    private List<Categoria> lista;
    private Integer idSelecionado = null;

    public TelaCategorias() {
        setTitle("Categorias");
        setSize(500, 400);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);

        JButton btnSalvar = new JButton("Salvar");
        JButton btnExcluir = new JButton("Excluir");
        JButton btnLimpar = new JButton("Limpar");

        JPanel topo = new JPanel(new FlowLayout(FlowLayout.LEFT));
        topo.add(new JLabel("Nome:"));
        topo.add(txtNome);
        topo.add(btnSalvar);
        topo.add(btnExcluir);
        topo.add(btnLimpar);

        tabela.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        add(topo, BorderLayout.NORTH);
        add(new JScrollPane(tabela), BorderLayout.CENTER);

        btnSalvar.addActionListener(e -> salvar());
        btnExcluir.addActionListener(e -> excluir());
        btnLimpar.addActionListener(e -> limpar());
        tabela.getSelectionModel().addListSelectionListener(e -> {
            int linha = tabela.getSelectedRow();
            if (!e.getValueIsAdjusting() && linha >= 0) {
                Categoria c = lista.get(linha);
                idSelecionado = c.getId();
                txtNome.setText(c.getNome());
            }
        });

        carregar();
    }

    private void carregar() {
        try {
            lista = dao.listar();
            modelo.setRowCount(0);
            for (Categoria c : lista) {
                modelo.addRow(new Object[]{c.getId(), c.getNome()});
            }
        } catch (SQLException ex) {
            erro(ex);
        }
    }

    private void salvar() {
        String nome = txtNome.getText().trim();
```

```
        if (nome.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Informe o nome da categoria.");
            return;
        }
        try {
            if (idSelecionado == null) {
                dao.inserir(new Categoria(0, nome));
            } else {
                dao.atualizar(new Categoria(idSelecionado, nome));
            }
            limpar();
            carregar();
        } catch (SQLException ex) {
            erro(ex);
        }
    }

    private void excluir() {
        if (idSelecionado == null) {
            JOptionPane.showMessageDialog(this, "Selecione uma categoria na tabela.");
            return;
        }
        int r = JOptionPane.showConfirmDialog(this, "Excluir a categoria selecionada?",
                "Confirmar", JOptionPane.YES_NO_OPTION);
        if (r != JOptionPane.YES_OPTION) return;
        try {
            dao.excluir(idSelecionado);
            limpar();
            carregar();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(this,
                "Não foi possível excluir. A categoria pode ter produtos cadastrados.\n" + ex.getMessage());
        }
    }

    private void limpar() {
        idSelecionado = null;
        txtNome.setText("");
        tabela.clearSelection();
    }

    private void erro(SQLException ex) {
        JOptionPane.showMessageDialog(this, "Erro de banco de dados:\n" + ex.getMessage());
    }
}
```

**loja/visao/TelaProdutos.java**

```
package loja.visao;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.table.DefaultTableModel;
import loja.dao.CategoriaDAO;
import loja.dao.ProdutoDAO;
import loja.modelo.Categoria;
import loja.modelo.Produto;

public class TelaProdutos extends JFrame {

    private final ProdutoDAO produtoDAO = new ProdutoDAO();
    private final CategoriaDAO categoriaDAO = new CategoriaDAO();

    private final JTextField txtNome = new JTextField(15);
    private final JTextField txtPreco = new JTextField(8);
    private final JTextField txtQtd = new JTextField(6);
    private final JComboBox<Categoria> cbCategoria = new JComboBox<>();
    private final JTextField txtBusca = new JTextField(20);

    private final DefaultTableModel modelo = new DefaultTableModel(
            new Object[]{"ID", "Nome", "Preço", "Quantidade", "Categoria"}, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };
    private final JTable tabela = new JTable(modelo);
    private List<Produto> lista = new ArrayList<>();
    private Integer idSelecionado = null;

    public TelaProdutos() {
        setTitle("Produtos");
        setSize(850, 500);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);

        JButton btnSalvar = new JButton("Salvar");
        JButton btnExcluir = new JButton("Excluir");
        JButton btnLimpar = new JButton("Limpar");
        JButton btnBuscar = new JButton("Buscar");
        JButton btnTodos = new JButton("Mostrar todos");

        JPanel form = new JPanel(new FlowLayout(FlowLayout.LEFT));
        form.add(new JLabel("Nome:"));       form.add(txtNome);
        form.add(new JLabel("Preço:"));      form.add(txtPreco);
        form.add(new JLabel("Quantidade:")); form.add(txtQtd);
        form.add(new JLabel("Categoria:"));  form.add(cbCategoria);

        JPanel botoes = new JPanel(new FlowLayout(FlowLayout.LEFT));
        botoes.add(btnSalvar);
        botoes.add(btnExcluir);
        botoes.add(btnLimpar);

        JPanel busca = new JPanel(new FlowLayout(FlowLayout.LEFT));
        busca.add(new JLabel("Buscar por nome ou categoria:"));
        busca.add(txtBusca);
        busca.add(btnBuscar);
        busca.add(btnTodos);

        JPanel norte = new JPanel();
        norte.setLayout(new BoxLayout(norte, BoxLayout.Y_AXIS));
        norte.add(form);
        norte.add(botoes);
        norte.add(busca);
        tabela.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
```

```
        add(norte, BorderLayout.NORTH);
        add(new JScrollPane(tabela), BorderLayout.CENTER);

        btnSalvar.addActionListener(e -> salvar());
        btnExcluir.addActionListener(e -> excluir());
        btnLimpar.addActionListener(e -> limpar());
        btnBuscar.addActionListener(e -> carregarProdutos(txtBusca.getText()));
        txtBusca.addActionListener(e -> carregarProdutos(txtBusca.getText()));
        btnTodos.addActionListener(e -> { txtBusca.setText(""); carregarProdutos(""); });

        tabela.getSelectionModel().addListSelectionListener(e -> {
            int linha = tabela.getSelectedRow();
            if (!e.getValueIsAdjusting() && linha >= 0) preencher(lista.get(linha));
        });

        // Recarrega as categorias quando a janela volta ao foco
        // (categorias novas cadastradas na outra tela aparecem aqui)
        addWindowListener(new WindowAdapter() {
            @Override public void windowActivated(WindowEvent e) { carregarCategorias(); }
        });

        carregarCategorias();
        carregarProdutos("");
    }

    private void carregarCategorias() {
        try {
            Categoria atual = (Categoria) cbCategoria.getSelectedItem();
            cbCategoria.removeAllItems();
            for (Categoria c : categoriaDAO.listar()) {
                cbCategoria.addItem(c);
                if (atual != null && c.getId() == atual.getId()) cbCategoria.setSelectedItem(c);
            }
        } catch (SQLException ex) {
            erro(ex);
        }
    }

    private void carregarProdutos(String busca) {
        try {
            lista = produtoDAO.listar(busca);
            modelo.setRowCount(0);
            for (Produto p : lista) {
                modelo.addRow(new Object[]{
                    p.getId(), p.getNome(), String.format("R$ %.2f", p.getPreco()),
                    p.getQuantidade(), p.getCategoriaNome()});
            }
        } catch (SQLException ex) {
            erro(ex);
        }
    }

    private void preencher(Produto p) {
        idSelecionado = p.getId();
        txtNome.setText(p.getNome());
        txtPreco.setText(String.format("%.2f", p.getPreco()).replace(",", "."));
        txtQtd.setText(String.valueOf(p.getQuantidade()));
        for (int i = 0; i < cbCategoria.getItemCount(); i++) {
            if (cbCategoria.getItemAt(i).getId() == p.getCategoriaId()) {
                cbCategoria.setSelectedIndex(i);
                break;
            }
        }
    }

    private void salvar() {
        String nome = txtNome.getText().trim();
        Categoria cat = (Categoria) cbCategoria.getSelectedItem();
        if (nome.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Informe o nome do produto.");
            return;
        }
        if (cat == null) {
            JOptionPane.showMessageDialog(this, "Cadastre ao menos uma categoria antes.");
            return;
        }
        double preco;
        int qtd;
        try {
            preco = Double.parseDouble(txtPreco.getText().trim().replace(",", "."));
            qtd = Integer.parseInt(txtQtd.getText().trim());
            if (preco < 0 || qtd < 0) throw new NumberFormatException();
```

```
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Preço e quantidade devem ser números válidos (não negativos).");
            return;
        }
        try {
            Produto p = new Produto(idSelecionado == null ? 0 : idSelecionado,
                                    nome, preco, qtd, cat.getId(), cat.getNome());
            if (idSelecionado == null) produtoDAO.inserir(p);
            else produtoDAO.atualizar(p);
            limpar();
            carregarProdutos(txtBusca.getText());
        } catch (SQLException ex) {
            erro(ex);
        }
    }

    private void excluir() {
        if (idSelecionado == null) {
            JOptionPane.showMessageDialog(this, "Selecione um produto na tabela.");
            return;
        }
        int r = JOptionPane.showConfirmDialog(this, "Excluir o produto selecionado?",
                "Confirmar", JOptionPane.YES_NO_OPTION);
        if (r != JOptionPane.YES_OPTION) return;
        try {
            produtoDAO.excluir(idSelecionado);
            limpar();
            carregarProdutos(txtBusca.getText());
        } catch (SQLException ex) {
            erro(ex);
        }
    }

    private void limpar() {
        idSelecionado = null;
        txtNome.setText("");
        txtPreco.setText("");
        txtQtd.setText("");
        tabela.clearSelection();
        if (cbCategoria.getItemCount() > 0) cbCategoria.setSelectedIndex(0);
    }

    private void erro(SQLException ex) {
        JOptionPane.showMessageDialog(this, "Erro de banco de dados:\n" + ex.getMessage());
    }
}
```

### 5.5 Classe principal

**loja/Main.java**

```
package loja;

import javax.swing.SwingUtilities;
import loja.visao.TelaPrincipal;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new TelaPrincipal().setVisible(true));
    }
}
```

## 6. Como o sistema funciona

- **Inserir ou atualizar:** a tela guarda o `idSelecionado`. Se for *null*, o botão Salvar chama `inserir()`; se tiver valor, chama `atualizar()`. Ao clicar em uma linha da tabela, o formulário é preenchido e o id é guardado.
- **Trocar a categoria na edição:** ao selecionar o produto, o JComboBox seleciona a categoria atual; mudando o combo e salvando, o UPDATE grava o novo `categoria_id`.
- **Busca:** `WHERE p.nome LIKE ? OR c.nome LIKE ?` com `JOIN` em Categorias. Texto vazio gera o filtro "%%", que lista tudo.
- **Combo sempre atualizado:** o `WindowListener` (windowActivated) recarrega as categorias quando a janela de produtos volta ao foco, então categorias novas aparecem sem reabrir a tela.
- **Excluir categoria com produtos:** a chave estrangeira bloqueia o DELETE e a tela mostra uma mensagem.

- **toString() em Categoria:** é ele que define o texto exibido no JComboBox.
- **try-with-resources:** fecha Connection, PreparedStatement e ResultSet automaticamente.
- **Telas com DISPOSE_ON_CLOSE:** fechar Produtos ou Categorias não encerra o programa; só a TelaPrincipal usa EXIT_ON_CLOSE.

# 7. Menus sem criar botões manualmente

Há dois caminhos: o **designer visual** do NetBeans ou um **JMenuBar por código**.

## 7.1 Pelo designer visual (arrastar e soltar)

- No pacote `loja.visao`: botão direito > *New* > *JFrame Form* (chame de TelaPrincipal; apague antes a versão por código, se existir).
- Na aba **Design**, abra a paleta (Ctrl+Shift+8) e arraste um **Menu Bar** (Swing Menus) para o topo da janela.
- No **Navigator** (Ctrl+7), botão direito em `jMenu1` > *Add From Palette* > *Menu Item*, uma vez para cada item (Produtos, Categorias, Sair).
- Duplo clique no texto de cada item para renomear (ex.: "Cadastros" > "Produtos").
- Botão direito no item > *Events* > *Action* > *actionPerformed*; o NetBeans cria o método e você escreve só a linha que abre a tela:

```
private void miProdutosActionPerformed(java.awt.event.ActionEvent evt) {
    new TelaProdutos().setVisible(true);
}
```

O NetBeans gera sozinho o código de criação e posicionamento dos componentes (blocos cinza "Generated Code", que não podem ser editados). Vale também para as telas de Produtos e Categorias criadas como *JFrame Form*; o código gerado fica mais extenso.

## 7.2 Por código, com JMenuBar

Alternativa à TelaPrincipal com botões (substitui a classe da seção 5.4):

**loja/visao/TelaPrincipal.java (versão com menu)**

```
package loja.visao;

import javax.swing.JFrame;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;

public class TelaPrincipal extends JFrame {

    public TelaPrincipal() {
        setTitle("Controle de Estoque");
        setSize(600, 400);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JMenuItem miProdutos = new JMenuItem("Produtos");
        JMenuItem miCategorias = new JMenuItem("Categorias");
        JMenuItem miSair = new JMenuItem("Sair");

        miProdutos.addActionListener(e -> new TelaProdutos().setVisible(true));
        miCategorias.addActionListener(e -> new TelaCategorias().setVisible(true));
        miSair.addActionListener(e -> System.exit(0));

        JMenu menuCadastros = new JMenu("Cadastros");
        menuCadastros.add(miProdutos);
        menuCadastros.add(miCategorias);
        menuCadastros.addSeparator();
        menuCadastros.add(miSair);

        JMenuBar barra = new JMenuBar();
        barra.add(menuCadastros);
        setJMenuBar(barra);
    }
}
```

Para não repetir `new JMenuItem` + `addActionListener` a cada item, crie um método auxiliar na própria classe:

```
private JMenuItem item(String texto, Runnable acao) {
    JMenuItem mi = new JMenuItem(texto);
    mi.addActionListener(e -> acao.run());
    return mi;
}

// uso:
menuCadastros.add(item("Produtos", () -> new TelaProdutos().setVisible(true)));
menuCadastros.add(item("Categorias", () -> new TelaCategorias().setVisible(true)));
```

## 8. Dicas e erros comuns

- **No suitable driver:** falta adicionar o MySQL JDBC Driver em *Libraries*.
- **Import em vermelho:** confira o nome do pacote/classe; use Ctrl+Shift+I (Fix Imports).
- **Access denied / Communications link failure:** confira usuário, senha, porta (3306) e se o MySQL está ativo.
- **Unknown database 'loja':** o schema.sql ainda não foi executado.
- **Declaração de package:** o package de cada arquivo precisa bater com a pasta (ex.: `package loja.dao;` em loja/dao).
- **Exercício extra:** criar o pacote `loja.controle` com a validação e a conversão de texto para número, deixando em `visao` apenas a interface.